
FPGAir

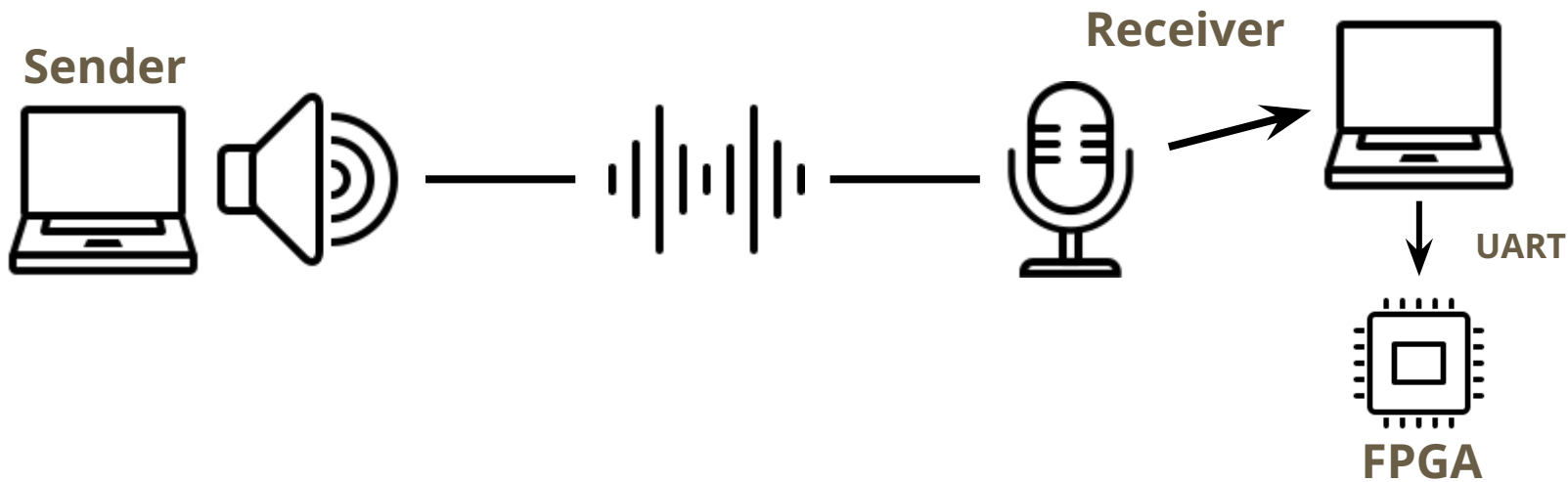
— Programmable Hardware Devices —

Emanuele Sarte

FPGAir - Sending data as Sound Waves

The **sender** encodes the message as sound and plays it with a speaker.

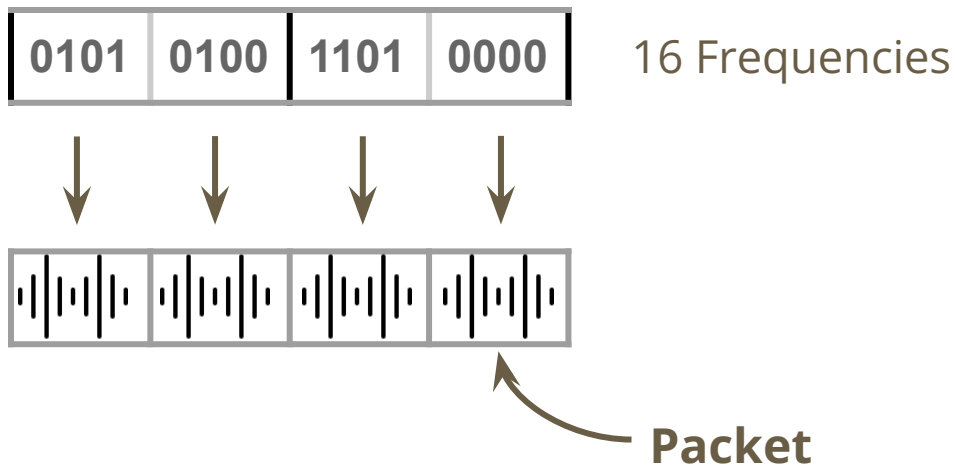
The **receiver** listens to it with a microphone and decodes the message.



Encoder

The message is encoded to bytes first.

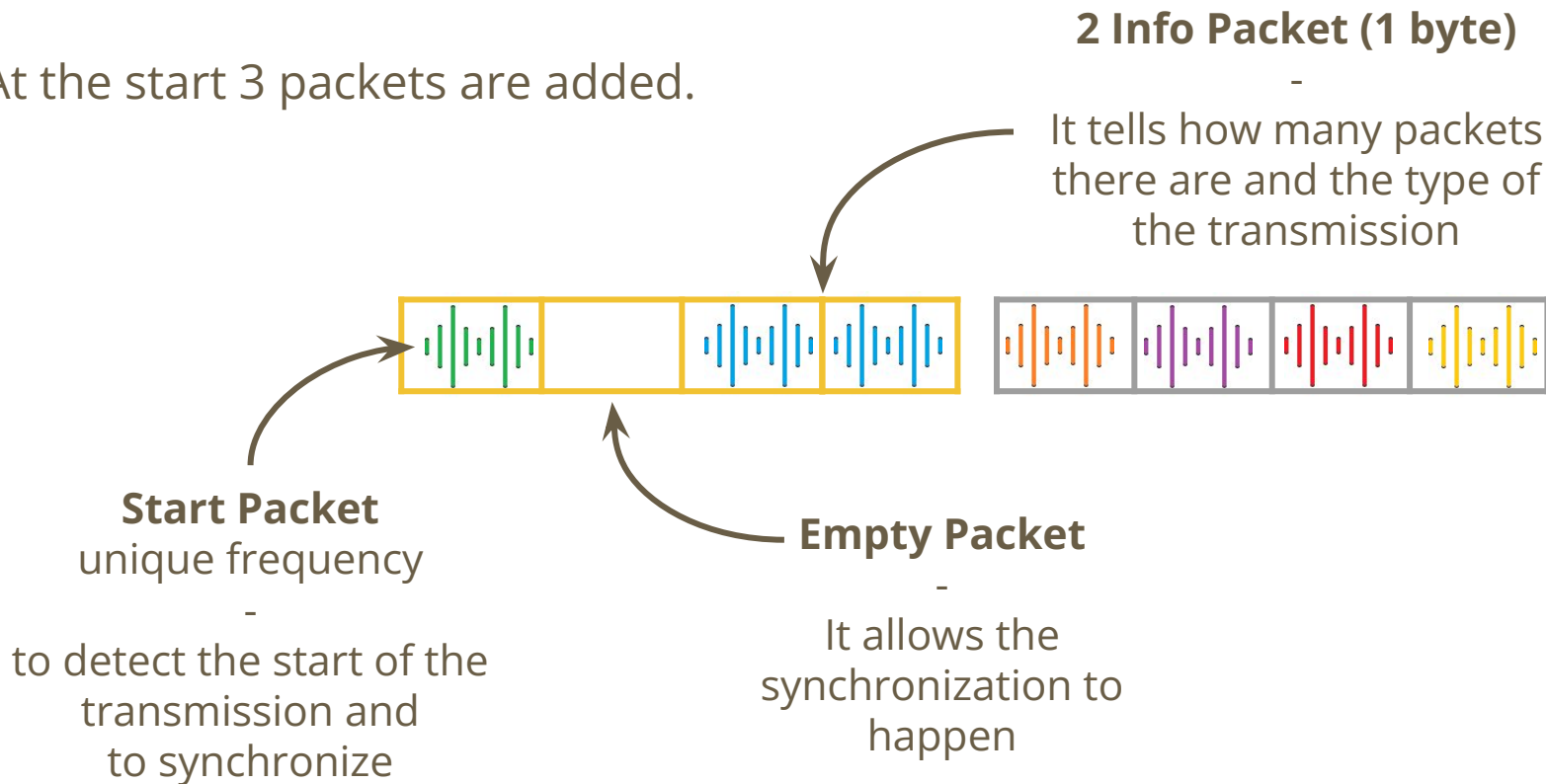
Each **half byte** is then encoded with a sound wave with a **specific frequency**.



A packet is composed of **1024 points**, sampled at **44100 Hz** $\approx 23\text{ms}$

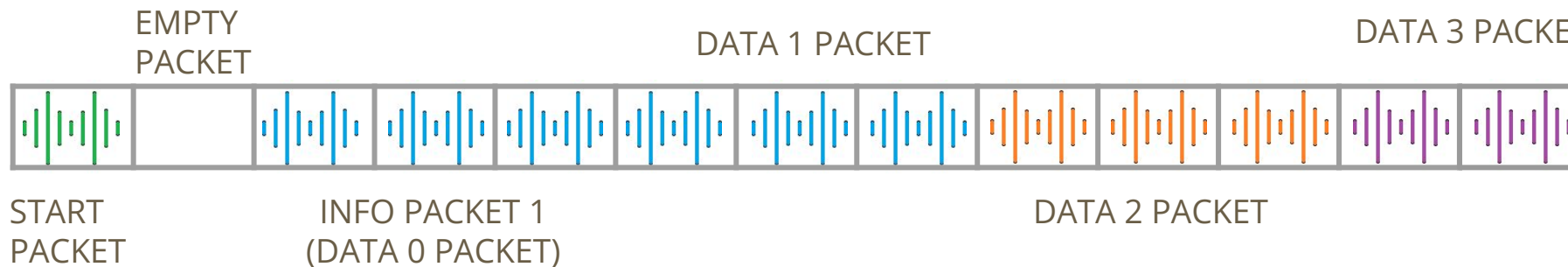
Encoder

At the start 3 packets are added.



Encoder

As **redundancy** each packet is tripled except for the first two



Decoder

The decoder detects the START PACKET and synchronizes itself with the sender. Then starts to decode the other packets.

It needs to be able **to decode the frequencies** of the packets. **How?**

The **frequency** we are interested in

↓

$$\mathcal{F} = \sum_j g(t_j) e^{i2\pi f t_j}$$

↑

The sound samples

$$\text{Re}(\mathcal{F}) = \sum_j^N \cos\left(2\pi f \cdot \frac{j}{F_s}\right) \cdot S_j$$

$$\text{Im}(\mathcal{F}) = \sum_j^N \sin\left(2\pi f \cdot \frac{j}{F_s}\right) \cdot S_j$$

Decoder

How to compute the sum?

$$\text{Re}(\mathcal{F}) = \sum_j^N \cos(2\pi f \cdot \frac{j}{F_s}) \cdot S_j$$

Problem

Solution

The cosine function

Use a lookup
table

The sine
function

$\sin(x) =$
 $\cos(x - \pi/2)$

Floating point
numbers

Just use
integers

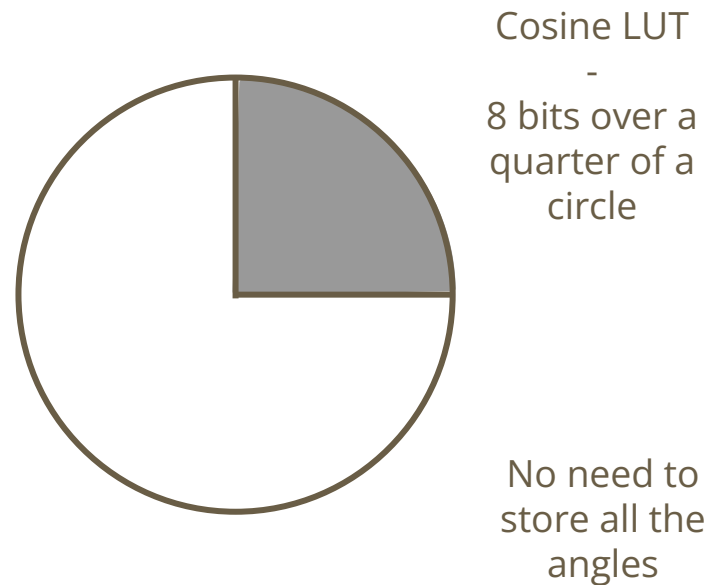
Decoder

Each quantity is given a precision in bits

$$\text{Re}(\mathcal{F}) = \sum_j^N \cos\left(2\pi f \cdot \frac{j}{F_s}\right) \cdot S_j$$

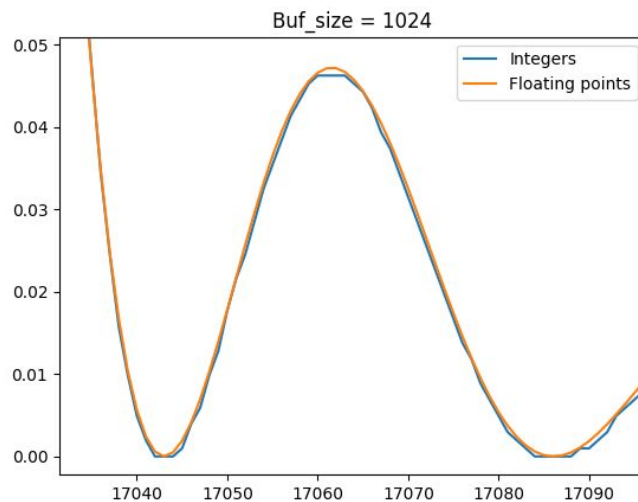
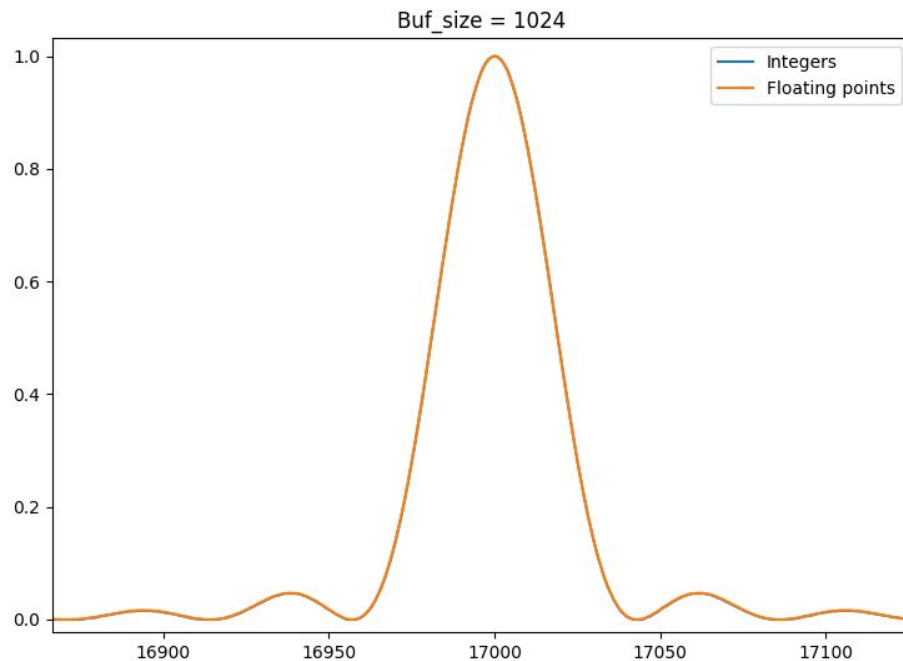
Diagram illustrating the equation components and their bit precision:

- $\cos\left(2\pi f \cdot \frac{j}{F_s}\right)$: 8 bits unsigned lookup table (indicated by a downward arrow from the text above).
- j : 1024 samples (indicated by an upward arrow from the text below).
- S_j : 24 bits signed integer (indicated by an upward arrow from the text below).



Decoder

Comparison between the floating point numbers and integers (normalized)

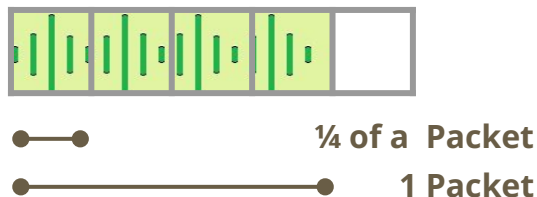


Decoder

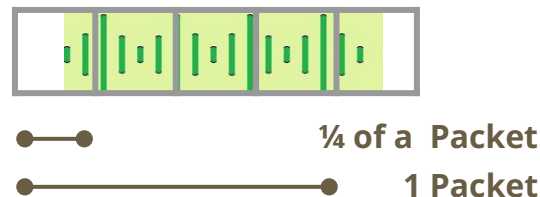
The detection of a transmission is pretty straightforward, just check if in the next batch of samples the starting frequency is above a threshold.

But how does the **synchronization** work?

Smaller batch to detect the starting frequency (a quarter of a packet = 256 points)



Already synchronized (by chance)



No synchronization (taking 5 subpackets)

Decoder

Calculating the magnitude of the starting and final packet allows us to calculate the **ratio of the shift** as:

$$R = \frac{A_f}{A_i + A_f}$$


The diagram shows a horizontal bar representing a packet. It is divided into four green rectangular sections by vertical lines. Each section contains a series of vertical bars of varying heights, resembling a waveform. Arrows point from the labels A_i and A_f in the equation above to the first and last green sections of the packet, respectively.

Shift of the last packet:

$$S = R * 256$$

We know the **next packet is empty** so we can drop samples until we are synchronized

Problem:

We have the magnitudes squared

Solution:

Just use a square root lookup table

Problem:

We need to perform a division

Solution:

Just breakdown it in more clock cycles

(Between two samples there are around 2200 clock cycles)

Decoder

To decode a packet, the decoder calculates the magnitude for all the 16 frequencies and chooses the biggest one. Once three packets are collected it runs the '**redundancy check**':

A	B	C	$A(B+C) + BC$
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

0	0	1	0	Packet 1
0	1	1	0	Packet 2
0	0	1	0	Packet 3
0	0	1	0	Final Packet

Decoder

Then every two 'final packet' it combines them into one byte and sends it back to the PC with the UART.

The decoder knows when the transmission ends because the first data packet contains the length of the transmission.